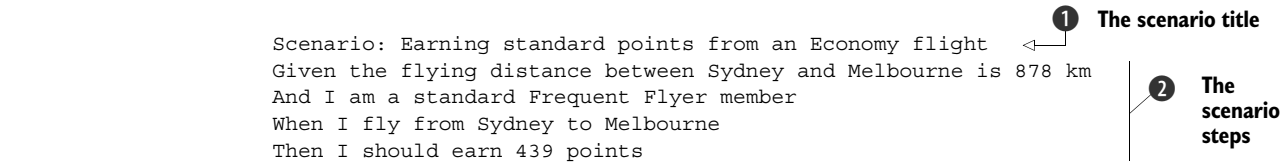
For the rest of this chapter, we'll take a closer look at what BDD-style automation looks like on a variety of platforms, pointing out the specificities of each platform. But before we do this, we need to discuss some general principles that will apply no matter what tool you choose.

6.1 Introduction to automating scenarios

Before we look at specific tools, let's go through the basics. In chapter 5, you saw how to describe requirements in terms of plain-text scenarios like the following:




```
Scenario: Earning standard points from an Economy flight
Given the flying distance between Sydney and Melbourne is 878 km
And I am a standard Frequent Flyer member
When I fly from Sydney to Melbourne
Then I should earn 439 points
```

This scenario is just loosely structured text. It explains what requirement you're trying to illustrate ❶ and how you intend to demonstrate that your application fulfills this requirement ❷. But how you actually perform each step will depend on your application and on how you decide to interact with it.

For example, the preceding scenario is made up of four steps ❷. Each of these steps needs to interact with the application to prepare the test environment, perform the action under test, and check the results. For example, consider the first `Given` step:

```
Given the flying distance between Sydney and Melbourne is 878 km
```

Here you need to configure a test database to provide the correct distance between Sydney and Melbourne. You could do this in many ways: you may want to inject data directly into a test database, call a web service, or manipulate a user interface. The text describes *what* you intend to do, but *how* you do this will depend on the nature of the application and on your technical choices.

The other steps are similar. For example, the first `When` step describes the action you're testing:

```
When I fly from Sydney to Melbourne
```

Again, this step describes *what* you want to do. You want to record a flight from Sydney to Melbourne so that you can check how many points the member earns. But *how* you do this requires more knowledge about your application and its architecture.

Tools like JBehave and Cucumber can't turn a text scenario into an automated test by themselves; they need your help. You need a way to tell your testing framework what each of these steps means in terms of your application, and how it must manipulate or query your application to perform its task. This is where *step definitions* come into play.

6.1.1 Step definitions interpret the steps

Step definitions are essentially bits of code that interpret the text in feature files and know what to do for each step (see figure 6.2).